Computational Statistics and Simulation: Section 5. Maximum likelihood estimation and EM algorithm

Master in Mathematics and Applications

Isabel Pereira,
isabel.pereira@ua.pt

Universidade de Aveiro,
2025/26

Section 5: Maximum likelihood estimation and EM algorithm

1. MLE-computation in R programme
   - Introduction
   - unidimensional estimation in R
   - multidimensional estimation in R
   - some remarks

2. Expectation-Maximization algorithm

1. MLE-Computation in R programme: Introduction

- MLE - Maximum Likelihood Estimation

  is a estimation method of the parameters of distribution

$$f(x|\theta), \theta \in \Theta \subseteq \mathbf{R}^k$$

- given the sample $\mathbf{x} = (x_1, \ldots, x_n)$, the method calculates the parameter value that maximizes the likelihood $L(\theta|\mathbf{x})$

$$L(\theta|\mathbf{x}) = \prod_{i=1}^{n} f(x_i|\theta) = f(x_1, \ldots, x_n|\theta)$$

- Maximum Likelihood Estimate (MLE) of θ, given the observed sample $\mathbf{x} = (x_1, \ldots, x_n)$

$$\hat{\theta} = \arg\max L(\theta|\mathbf{x}), i.e., L(\hat{\theta}|\mathbf{x}) = \max_{\theta \in \Theta} L(\theta|\mathbf{x})$$

1. MLE-Computation in R programme: Introduction

- Since log function is monotonic, in practice we consider Log-likelihood, and search the values of θ that maximizes it, i.e,

$$\hat{\theta} = \arg\max(\log L(\theta|\mathbf{x}))$$

- If $k = 1$ (one parameter), under regularity conditions, the maximum is obtained as a solution of the likelihood equation

$$\frac{d}{d\theta}\log L(\theta|\mathbf{x}) = L^*(\theta|\mathbf{x}) = 0,$$

with

$$L^*(\theta|\mathbf{x}) := \log L(\theta|\mathbf{x}) = \sum_{i=1}^{n} \log f(x_i|\theta)$$

Remarks:

1. Sometimes there is no explicit analytical solution for the solution and there is a need to resort to computational methods to obtain it.
2. The use of L^* comes, on the one hand, to simplify the task of obtaining the likelihood equation and, on the other hand, to smooth the behavior of computational iterations.

1. MLE-Computation in R: one-dimensional case

In practice how to proceed?

We can get the analytical solution. But sometimes this is not feasible and is needed some numerical procedures.

These are implemented in R programme:

- use mle command from R package stats4

  obtains the solution that minimizes a given function, so consider -log L

- use optimize command of R programme

  obtains the solution that maximizes a function, so consider log L

- use uniroot command of R programme

  obtains the zeros of a given function

Remark:

1. these numerical procedures demand a **one dimensional parameter**.

Example 1.1: MLE of exponential distribution
Let $X := (X_1, \ldots, X_n)$ a random sample from Exponential population with pdf

$$f(x|\theta) = \theta \mathrm{e}^{-\theta x}, x > 0 (\theta > 0)$$

Finding analytically the MLE of the parameter and using the mle functions from the stats4 R package, optimize function or uniroot considering the observed sample $(x_1, \ldots, x_n)$.

$$\frac{d}{d\theta} \log L(\theta|\mathbf{x}) = 0 \Leftrightarrow \frac{n}{\theta} - \sum_{i=1}^{n} x_i = 0$$

i.e., the estimator of θ is

$$\hat{\theta} = \frac{n}{\sum X_i} = \frac{1}{\bar{X}}$$

mle function in stats4 R package

```
## ----- (EMV usando mle) ----------------------------------
n=10;teta=2.5  ##  amostra  de n geradas EXP(valor médio = 1/theta)
x <- rexp(n,rate=teta) # --- theta=rate -----
mlogL <- function(theta) { # - log-verosimilhança , ie, -L*
   return( - (length(x) * log(theta) - theta * sum(x)))
}
library(stats4)
fit <- mle(mlogL, start=list(theta=1))
summary(fit)
# ----------------------------------------
Maximum likelihood estimation

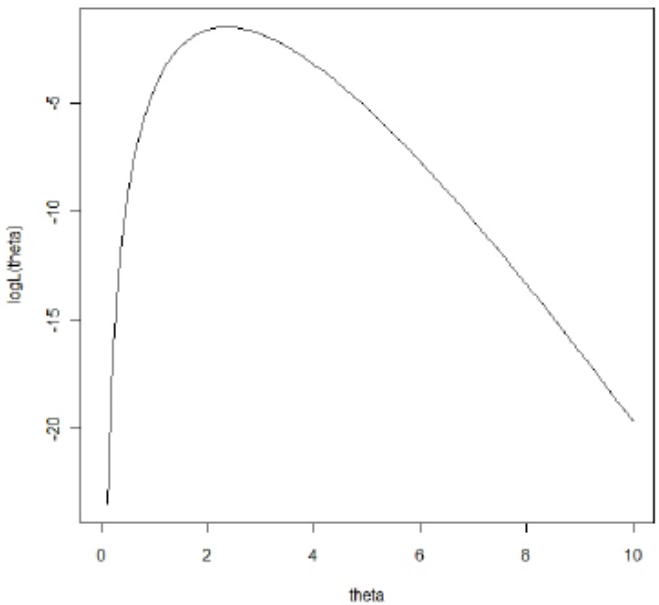
Call:
mle(minuslogl = mlogL, start = list(theta = 1))

Coefficients:
      Estimate Std. Error
theta 2.375644  0.7512446

-2 log L: 2.694624
>     cat("o inverso da média é =",1/mean(x))
o inverso da média é = 2.375645>
```

optimize command in R programme

```
## ----- (EMV usando optimize) ----------------------------------
 n=10;teta=2.5  ##  amostra  de n geradas EXP(valor médio = 1/theta)
 x <- rexp(n,rate=teta) # --- theta=rate -----
 logL <- function(theta) { # log-verosimilhança , ie, L*
    return( length(x) * log(theta) - theta * sum(x))
 }
 theta <- seq(0, 10, .1)
 plot(theta,logL(theta), type = "l" )
```



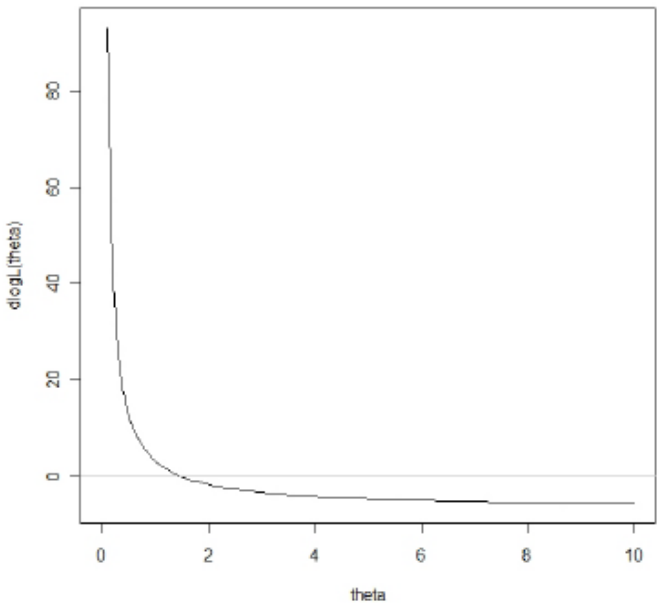

```
> optimize(logL,lower=2,upper=6,maximum=T)
$maximum
[1] 2.342686

$objective
[1] -1.487018

>  cat("o inverso da média é =",1/mean(x),
o inverso da média é = 2.342686>
```

uniroot command in R programme

```
## ----- (EMV usando uniroot) ----------------------------------
n=10;teta=2.5  ##  amostra  de n geradas EXP(valor médio = 1/theta)
x <- rexp(n,rate=teta) # --- theta=rate -----
dlogL <- function(theta) {#equação de verosimilhança,  dL*/dtheta=0
   return( length(x) /theta - sum(x))
}
theta <- seq(0, 10, .1)
plot(theta,dlogL(theta), type = "l" );abline(h=0, col="grey")
```




```
>  uniroot(dlogL, lower = 1, upper =3)
$root
[1] 1.486985

$f.root
[1] -1.196851e-05

$iter
[1] 6

$estim.prec
[1] 6.103516e-05

> cat("o inverso da média é =",1/mean(x),'
o inverso da média é = 1.486980
```

Example 1.2: MLE of $Gamma(\alpha, \lambda)$ distribution
Let $\mathbf{x} := (x_1, \ldots, x_n)$ be an observed sample from Gamma(α, λ) population with pdf

$$f(x|\alpha, \lambda) = \frac{\lambda^\alpha}{\Gamma(\alpha)} x^{\alpha-1} \mathrm{e}^{-\lambda x}, x > 0 (\alpha > 0, \lambda > 0)$$

Find the MLE of (α, λ) using the uniroot function of R, from an observed sample of n values generated from Gamma(α, λ).

$$\log L(\alpha, \lambda|\mathbf{x}) = n\alpha \log\lambda - n\log\Gamma(\alpha) + (\alpha - 1) \sum_{i=1}^{n} \log x_i - \lambda \sum_{i=1}^{n} x_i$$

It is about optimizing a two-dimensional problem that will be reduced to a one-dimensional problem.

1. MLE-Computation in R: two-dimensional case

We have to obtain the solutions of the maximum likelihood equations system

$$\begin{cases} \frac{\partial}{\partial\lambda}\log L(\alpha,\lambda|\mathbf{x}) = \frac{n\alpha}{\lambda} - \sum_{i=1}^{n} x_i = 0 \\ \frac{\partial}{\partial\alpha}\log L(\alpha,\lambda|\mathbf{x}) = n\log\lambda - n\frac{\Gamma'(\alpha)}{\Gamma(\alpha)} + \sum_{i=1}^{n}\log(x_i) = 0 \end{cases}.$$

leading to

$$\begin{cases} \alpha = \lambda\bar{x} \\ \log\lambda + \frac{1}{n}\sum_{i=1}^{n}\log(x_i) = \psi(\lambda\bar{x}) \end{cases}.$$

where $\psi(t) = \frac{d}{dt}\log\Gamma(t) = \frac{\Gamma'(t)}{\Gamma(t)}$ (in R programme is denoted as digamma)

Since, 2nd equation only evolves one parameter we can apply uniroot command

uniroot command in R programme-gammma distribution

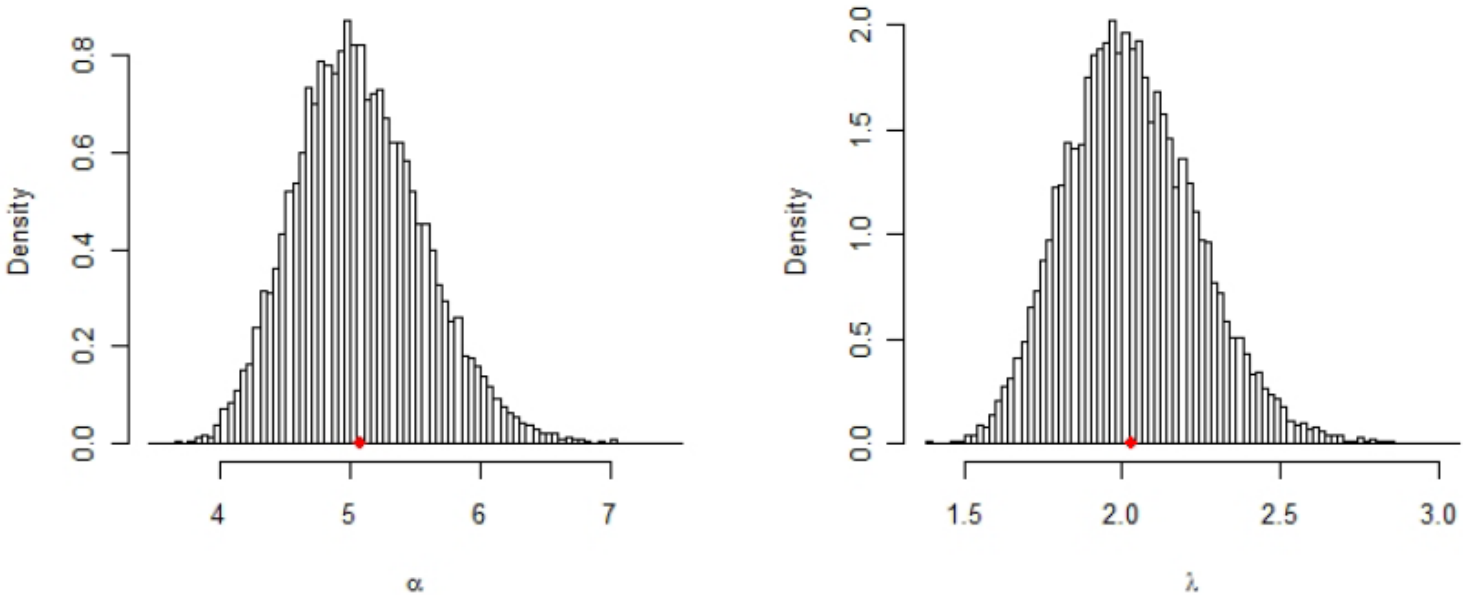
- Repetir N=20000 vezes a
- Geração de n=200 valores Gama($\alpha = 5, \lambda = 2$) e estimação EMV
- Fazer as médias das estimativas de α ,λ, # de iterações e histogramas.

```
## ------- (MLE: Gama(forma,taxa))
N <- 20000 ; est <- matrix(0, N, 3)
n <- 200 ; alfa <- 5 ;lambda <- 2
obj <- function(lambda, xbar, logx.bar) {
digamma(lambda * xbar) - logx.bar - log(lambda)
}
for (i in 1:N) {
x <- rgamma(n, shape=alfa, rate=lambda)
xbar <- mean(x)
logx.bar <- mean(log(x))
u <- uniroot(obj, lower = .001, upper = 10e5,
xbar = xbar, logx.bar = logx.bar)
lambda.hat <- u$root
iter <- u$iter
alfa.hat <- xbar * lambda.hat
est[i, ] <- c(alfa.hat, lambda.hat, iter)
}
ML <- colMeans(est)
    > ML
[1] 5.071133 2.030375 24.433650
```

1. MLE-Computation in R programme: two-dimensional case

uniroot command in R programme-gammma distribution

```
par(mfrow = c(1, 2))
    hist(est[, 1], breaks="scott", freq=FALSE,
        xlab=bquote(alpha), main="")
    points(ML[1], 0, cex=1.5, pch=20,col="red")
    hist(est[, 2], breaks="scott", freq=FALSE,
         xlab=bquote(lambda), main="")
    points(ML[2], 0, cex=1.5, pch=20,col="red")
```



1. MLE-Computation in R: two-dimensional case

We can, alternatively, get the optimal solution by considering as a **2 dimensional problem**.

In R, it can be used **optim command**.

- optim command minimizes a given function, so consider -log L
- it is needed to give some initial values

  usually we consider the moment method estimates
- to solve the optimization problem it is used some Newton-Raphson - type iterative methods, but more sophisticated, and it will be given some information concerning the convergence of the methods.

optim command in R programme-gammma distribution

In this case the function to be minimized is

$$L^*(\alpha,\lambda) = \log L(\alpha,\lambda) = n\alpha\log\lambda - n\log\Gamma(\alpha) + (\alpha-1)\sum_{i=1}^{n}\log(x_i) - \lambda\sum_{i=1}^{n}x_i$$

```
### ------(Gama(forma,taxa) com optim - optimização bidimensional )
LL <- function(theta, sx, slogx, n) {
 alfa <- theta[1]
 lambda <- theta[2]
 loglik <- n*alfa*log(lambda)- n*log(gamma(alfa)) + (alfa-1)*slogx -lambda*sx
 return(- loglik)
        }

    n <- 200
    alfa <- 5;    lambda <- 2
    x <- rgamma(n, shape=alfa, rate=lambda)
##------------  valores iniciais (1,1) -------------------
    optim(c(1,1), LL, sx=sum(x), slogx=sum(log(x)), n=n)
```

1. MLE-Computation in R: two-dimensional case

The correspondent output is

```
$par
[1] 5.654226 2.237523

$value
[1] 283.6476

$counts
function gradient
      77       NA

$convergence
[1] 0

$message
NULL
```

Remarks:

- the initial values were here $(\alpha, \lambda) = (1, 1)$, advisible m.m. estimates
- optim minimizes by default, so was used return (-log lik)
- par $\hookrightarrow$ solution of the optimization problem
- value $\hookrightarrow$ minimum value of the function
- convergence, message, $\hookrightarrow$ report eventual convergence problems

1. MLE-Computation in R: two-dimensional case

Replicating the procedure 20000 times, and calculating the correspondent means of the correspondent solutions

```
mlests <- replicate(20000, expr = {
  x <- rgamma(200, shape = 5, rate = 2)
  optim(c(1,1), LL, sx=sum(x), slogx=sum(log(x)), n=n)$par
  })
colMeans(t(mlests))
```

```
[1] 5.071161 2.030446
```

The produced estimates are very closed to input values for the shape $\alpha = 5 (\hat{\alpha} \approx 5.071)$ and rate $\lambda = 2 (\hat{\lambda} \approx 2.030)$

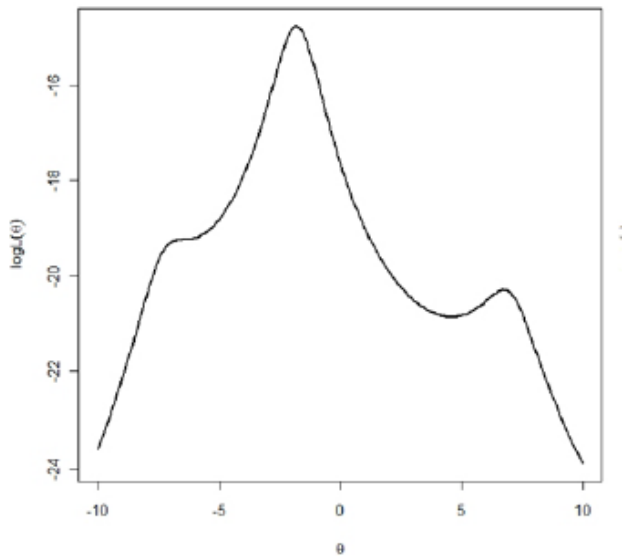
Remarks:

- replicate produced a matrix with 2 lines and 20000 columns and, therefore, the transposition was carried out, using colMeans(t(mlests)).
- Alternatively, one could have done the averages per row, using rowMeans(mlests)

1. MLE-Computation in R programme: some remarks

Remarks:

- It should be noted that numerical optimization methods for iterative max-likelihood equations must start from a sufficiently close approximation to the solution, as mentioned above.
- In fact, the behavior of likelihood can be "meandering"in the sense that there are several local maxima and not just one global maximum, which can lead to stability points far from the exact solution.



2. Expectation-Maximization (EM) algorithm

The EM Algorithm (Expectation Maximization) allows to calculate the Maximum Likelihood estimates in the presence of unobserved variables. This is a data augmentation method; the data observed are augmented with latent (unobserved) data in order to simplify the calculations.

Consider the following framework:

- $\mathbf{Y} \hookrightarrow$ vector that describes the observed variables
- $\mathbf{Z} \hookrightarrow$ vector that describes the non observed variables (called latent)
- $\theta \hookrightarrow$ parameter to estimate, using the joint distribution $(\mathbf{Y}, \mathbf{Z})$ - augmented data
- The likelihood is

$$L(\theta|\mathbf{y}, \mathbf{z}) = f(\mathbf{y}, \mathbf{z}|\theta)$$

2. Expectation-Maximization (EM) algorithm

The EM can be used in situations where the data structure suggest the existence

- missing data
- censored data
- mixture of distributions

The goal is estimate θ, given $\mathbf{Y} = \mathbf{y}$, maximizing

$$L(\theta|\mathbf{y}) = f(\mathbf{y}|\theta) = \int f(y, z|\theta)dz = \int L(\theta|y, z)dz$$

using the logarithm $\log L(\theta|\mathbf{y})$

2. Expectation-Maximization (EM) algorithm

Since the augmented sample $(\mathbf{Y}, \mathbf{Z}) \sim f(\mathbf{y}, \mathbf{z}|\theta)$, the conditional of $\mathbf{Z}|\theta, \mathbf{y})$ is

$$f(z|\theta, \mathbf{y}) = \frac{f(\mathbf{y}, \mathbf{z}|\theta)}{f(\mathbf{y}|\theta)}$$

and taking the logarithm, we obtain the following relation between

- the likelihood of the complete sample, $L^c(\theta|\mathbf{Z}, \mathbf{y})$
- and the likelihood of the observed data, $L(\theta|\mathbf{y})$

For any θ_0,

$$\log L(\theta|y) = E_{\theta_0}[\log L^c(\theta|\mathbf{Z}, \mathbf{y})] - E_{\theta_0}[\log f(\mathbf{Z}|\theta, \mathbf{y})]$$

where the expected value is calculated using the distribution of $\mathbf{Z}|\theta_0, \mathbf{y} \sim f(\mathbf{z}|\theta_0, \mathbf{y})$

We define the function Q as

$$Q(\theta|\theta_0, \mathbf{y}) := E_{\theta^0}[\log L^c(\theta|\mathbf{Z}, \mathbf{y})]$$

EM algorithm

input: an initial estimate $\theta^{(0)}$

output: sequence of $\theta^{(i)}, i = 1, 2, \ldots$ - iterated estimates for θ

1. Expectation step (E step)

$$Q(\theta|\hat{\theta}^{(i)}, \mathbf{y}) := E_{\theta^{(i)}}[\log L^c(\theta|\mathbf{Z}, \mathbf{y})]$$

(expected value using the distribution of $\mathbf{Z}|\theta^{(i)}, \mathbf{y}$)

2. Maximization step (M step):
Find the parameters that maximize

$$\theta^{(i+1)} = \arg\max Q(\theta^{(i)}, \mathbf{y})$$

do $i = i + 1$

until $\theta^{(i+1)} = \theta^{(i)}$ for a particular fixed precision

2. Expectation-Maximization (EM) algorithm

- The EM algorithm is a deterministic optimization technique, introduced by Dempster, Laird and Rubin (1977)
- It is shown that (Jensen inequality)

$$L(\hat{\theta}_{i+1}|\mathbf{y}) \geqslant L(\hat{\theta}_i|\mathbf{y}),$$

  whereby, under certain conditions, every boundary point of the EM sequence $\hat{\theta}_i$ is a stationarity point of $L(\theta|\mathbf{y})$ and though not necessarily the MLE or even the local maximum.
- This means that, in practice, one must run the EM algorithm for several randomly chosen starting points.
- This is the only element of randomness involved in the EM algorithm, since using the same starting point $\hat{\theta}_0$ leads to the same value.

References - Bibliography

- Dempster,A.P., Laird,N. M. and D. B. Rubin, D. B. (1977) Maximum Likelihood from Incomplete Data via the EM Algorithm. Journal of the Royal Statistical Society. Series B (Methodological)Vol. 39, No.1, pp. 1-38.